

SafePay — Risk-Adaptive Payment Protection System

Complete Project Blueprint

1. Problem Statement

Digital payment rails (UPI, IMPS, NEFT, RTGS) are optimized for speed, not reversibility. Two gaps exist today:

No differentiated protection. Existing "cooling-off" rules (e.g., new-beneficiary holds on NEFT/RTGS) apply the *same* fixed delay to every transaction regardless of actual risk. A ₹500 payment to a new beneficiary gets the same treatment as a ₹5,00,000 payment to a new beneficiary — either both are needlessly delayed, or neither is protected enough.

No orchestrated decision layer. Banks already have authentication, fraud monitoring, and post-transaction dispute/recovery processes — but nothing that takes the *signals* these systems already produce (new beneficiary, unusual amount, new device, odd hour) and turns them into a single, explainable, real-time decision: *release instantly, protect briefly, or hold for approval*.

The result: users who make an honest mistake (wrong account, fat-fingered amount) have no way to catch it before the money is gone, and corporate users have no lightweight way to add a second set of eyes to unusually large payments without a full manual approval process for *everything*.

2. Our Solution

SafePay is not a fraud-detection system and not a payment rail. It's a **policy/orchestration layer** that sits between "payment authorized" and "payment settled," and answers one question per transaction:

How much friction does this specific payment deserve before it becomes irreversible?



Payment → Risk Signals → Risk Score → Policy Decision → Release / Protect / Hold

LOW risk → instant settlement (no friction)

MEDIUM risk → short protection window (e.g. 10s) — user can Cancel or Release

HIGH risk → longer protection window (e.g. 60s) — user can Cancel or Release

VERY HIGH risk → hard hold — routed to a maker-checker approval queue instead of a timer

The system is built as a **simulated transaction engine** with its own internal ledger and state machine — it does not claim to intercept or reverse a live UPI/IMPS/NEFT transaction on the real payment rails (which is neither technically possible nor within a student/portfolio project's authority). This is the honest and correct scope, and it's what makes the project technically credible.

3. Feature List

A. Core Transaction Engine

Create payment (beneficiary, amount, remarks)

Simulated authentication step (PIN/OTP mock)

Transaction state machine: `CREATED → AUTHORIZED → PROTECTED → CANCELLED / RELEASED → SETTLED → DISPUTED → RECOVERED`

Server-authoritative countdown timer (never trust client clock)

Auto-release job when protection window expires

B. Risk Engine

Rule-based, weighted risk scoring (explainable, not a black box)

Signals: beneficiary age, amount vs. historical average, new device, new location, unusual time of day, transaction velocity

Risk level classification: LOW / MEDIUM / HIGH / VERY_HIGH

Per-transaction risk explanation ("Protected because: new beneficiary + amount 9x your average")

C. Customer-Facing Features

Add/manage beneficiaries (with cooling-off metadata)

Initiate payment with live risk feedback before confirming

Protection window UI: countdown + Cancel / Release buttons

Configurable personal protection preferences (protect above ₹X, always protect new beneficiaries, custom window length)

Transaction history with status and risk explanation

Raise a dispute for a settled transaction

D. Enterprise / Maker-Checker Mode

Role-based accounts: Maker, Checker, Admin

High-value or VERY_HIGH-risk transactions routed to a checker queue instead of a timer

Checker approves/rejects with remarks

Full audit trail of who created, approved, or rejected a transaction

E. Admin / Bank-Ops Features

Configure risk weights and thresholds per risk level

Configure default protection window durations

View all transactions, override/force-release, view audit logs

Dashboard: transactions by status, average protection time, cancellation rate, dispute rate

F. Dispute & Recovery (post-settlement)

Raise dispute → status tracking (Open → In Review → Resolved/Rejected)

Simulated recovery workflow, kept clearly separate from the pre-settlement protection layer

G. Auth & Security

JWT-based authentication, Spring Security

Role-based access control (CUSTOMER, MAKER, CHECKER, ADMIN)

Audit logging on every state transition

H. Notifications

In-app + simulated email/SMS on: beneficiary added, payment protected, window expiring, released, held for approval, dispute update

4. Complete Project Flow (Start to End)

Register / Login — user signs up, gets JWT token.

Add beneficiary — system records `added_at` timestamp (used later for the "new beneficiary" risk signal).

Initiate payment — user selects beneficiary, enters amount, remarks.

Authenticate — simulated PIN/OTP step confirms intent.

Risk engine runs — computes signals (beneficiary age, amount deviation, device/location, time of day) → weighted score → risk level.

Policy decision applied:

LOW → transaction moves straight to `SETTLED`.

MEDIUM/HIGH → transaction enters `PROTECTED` with a `protected_until` timestamp; frontend shows a live countdown with Cancel/Release actions. Server is the source of truth — a scheduled job also auto-transitions expired windows to `RELEASED → SETTLED` even if the user never touches the UI.

VERY_HIGH (or an enterprise rule like "> ₹10L requires checker") → transaction enters `PROTECTED` with `requires_checker = true` and is pushed to the checker's approval queue instead of a timer.

Checker review (if applicable) — checker approves (→ `RELEASED → SETTLED`) or rejects (→ `CANCELLED`), with remarks logged.

Notifications fire at each transition.

Post-settlement — if the user later realizes an error, they raise a **dispute**, which is handled as a separate workflow (`SETTLED → DISPUTED → RECOVERED/REJECTED`), clearly distinct from the pre-settlement protection flow.

Admin/ops can view every transaction, its full risk-signal breakdown, and the audit trail at any point.

5. Data Model

Entities & Key Fields

`app_user`

Column	Type	Notes
id	NUMBER (PK)	Oracle sequence-generated
name	VARCHAR2	
email	VARCHAR2 (unique)	
phone	VARCHAR2	
password_hash	VARCHAR2	BCrypt
role	VARCHAR2	CUSTOMER / MAKER / CHECKER / ADMIN
created_at	TIMESTAMP	

account

Column	Type	Notes
id	NUMBER (PK)	
user_id	NUMBER (FK → app_user)	
account_number	VARCHAR2	
balance	NUMBER(18,2)	
avg_txn_amount	NUMBER(18,2)	rolling average, used as a risk signal
created_at	TIMESTAMP	

beneficiary

Column	Type	Notes
id	NUMBER (PK)	
owner_account_id	NUMBER (FK → account)	
beneficiary_name	VARCHAR2	
beneficiary_account_number	VARCHAR2	
ifsc_code	VARCHAR2	
added_at	TIMESTAMP	drives "new beneficiary" risk signal
is_trusted	CHAR(1)	Y/N, set after N successful payments
txn_count	NUMBER	

transaction

Column	Type	Notes
id	NUMBER (PK)	
from_account_id	NUMBER (FK → account)	
beneficiary_id	NUMBER (FK → beneficiary)	
amount	NUMBER(18,2)	
status	VARCHAR2	CREATED / AUTHORIZED / PROTECTED / CANCELLED / RELEASED / SETTLED / DISPUTED / RECOVERED
risk_score	NUMBER(5,2)	
risk_level	VARCHAR2	LOW / MEDIUM / HIGH / VERY_HIGH
protection_window_seconds	NUMBER	nullable — null for LOW
protected_until	TIMESTAMP	nullable, server-authoritative expiry
requires_checker	CHAR(1)	Y/N
device_id	VARCHAR2	
location	VARCHAR2	
remarks	VARCHAR2	
created_at / updated_at	TIMESTAMP	
risk_signal_log (audit trail / explainability)		

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	NUMBER (FK)	
signal_name	VARCHAR2	e.g. NEW_BENEFICIARY, AMOUNT_DEVIATION
signal_value	VARCHAR2	
weight	NUMBER(5,2)	
contribution_score	NUMBER(5,2)	
evaluated_at	TIMESTAMP	
protection_policy (admin-configurable)		

Column	Type	Notes
id	NUMBER (PK)	
risk_level	VARCHAR2	
default_window_seconds	NUMBER	
requires_checker	CHAR(1)	
min_score / max_score	NUMBER(5,2)	thresholds

user_preference

Column	Type	Notes
id	NUMBER (PK)	
user_id	NUMBER (FK)	
protect_above_amount	NUMBER(18,2)	
protect_new_beneficiary	CHAR(1)	
custom_window_seconds	NUMBER	nullable
confirm_above_amount	NUMBER(18,2)	

maker_checker_approval

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	NUMBER (FK)	
maker_id	NUMBER (FK → app_user)	
checker_id	NUMBER (FK → app_user, nullable)	
status	VARCHAR2	PENDING / APPROVED / REJECTED
remarks	VARCHAR2	
decided_at	TIMESTAMP	
<code>dispute</code>		

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	NUMBER (FK)	
raised_by	NUMBER (FK → app_user)	
reason	VARCHAR2	
status	VARCHAR2	OPEN / IN_REVIEW / RESOLVED / REJECTED
created_at / resolved_at	TIMESTAMP	
<code>notification</code>	id, user_id, transaction_id, type, message, channel, sent_at	
<code>audit_log</code>	id, entity_type, entity_id, action, performed_by, timestamp, details	

Relationships


app_user (1) — (*) account
account (1) — (*) beneficiary
account (1) — (*) transaction [from_account]
beneficiary (1) — (*) transaction
transaction (1) — (*) risk_signal_log
transaction (1) — (0..1) maker_checker_approval
transaction (1) — (0..1) dispute
app_user (1) — (1) user_preference

6. API Endpoints

Auth



POST /api/auth/register

POST /api/auth/login

POST /api/auth/refresh

Accounts



GET /api/accounts/{id}

GET /api/accounts/{id}/balance

Beneficiaries



POST /api/beneficiaries

GET /api/beneficiaries?accountId={id}

DELETE /api/beneficiaries/{id}

Transactions (core engine)



POST /api/transactions/initiate → runs risk engine, returns decision + window

POST /api/transactions/{id}/authenticate → simulated OTP/PIN step

GET /api/transactions/{id}

GET /api/transactions?accountId={id}&status={status}

POST /api/transactions/{id}/cancel

POST /api/transactions/{id}/release → manual early release, optional

GET /api/transactions/{id}/risk-explanation

Maker-Checker



GET /api/approvals/pending?checkerId={id}

POST /api/approvals/{id}/approve

POST /api/approvals/{id}/reject

Preferences



GET /api/preferences/{userId}
PUT /api/preferences/{userId}

Disputes


POST /api/disputes
GET /api/disputes/{id}
PUT /api/disputes/{id}/status

Admin


GET /api/admin/policies
PUT /api/admin/policies/{riskLevel}
GET /api/admin/audit-logs
GET /api/admin/dashboard/stats

Notifications


GET /api/notifications?userId={id}

Real-time


WS /ws/transactions/{id}/status → pushes live countdown + state changes (STOMP over WebSocket)

7. Tech Stack Mapping

Layer	Technology	Notes
Backend framework	Spring Boot 3.x (Java 17)	Spring MVC controllers run on the Servlet API under the hood — covers your "servlets" requirement natively; no need for raw <code>HttpServlet</code> classes unless you want one for a specific simulated webhook.
Security	Spring Security + JWT	Role-based access (CUSTOMER/MAKER/CHECKER/ADMIN)
Persistence	Spring Data JPA (Hibernate) + Oracle DB	Use <code>@GeneratedValue(strategy = GenerationType.SEQUENCE)</code> with Oracle sequences; <code>@Entity</code> classes map directly to the tables above

Layer	Technology	Notes
Scheduling	Spring <code>@Scheduled</code>	Periodic job scans <code>PROTECTED</code> transactions where <code>protected_until < now()</code> and auto-transitions them — this is what makes the server (not the browser) authoritative over the timer
Real-time updates	Spring WebSocket (STOMP)	Pushes countdown/status updates to the frontend so the UI doesn't have to poll
Frontend	Oracle JET (ojet)	Module-based UI (<code>oj-module</code>), Knockout-driven data binding, <code>oj-c-button</code> / <code>oj-progress-status</code> type components for the countdown + Cancel/Release UI; REST calls via <code>fetch</code> / <code>oj-rest-dataprovider</code>
Build tools	Maven (backend), ojet-cli (frontend scaffolding)	
Testing	JUnit 5 + Mockito	Especially important for the risk-scoring function — pure logic, easy to unit test with table-driven test cases
Deployment	Spring Boot embedded Tomcat	Simplifies deployment; Oracle DB connection via Oracle JDBC driver (<code>ojdbc11</code>)

One critical implementation note

Never let the frontend own the countdown as the source of truth. The **server stores** `protected_until`; the Oracle JET UI just displays `protected_until - now()` and re-syncs on each WebSocket tick. The actual `PROTECTED → RELEASED` transition happens server-side (via the scheduled job), so even if a user closes the tab or manipulates their local clock, the outcome is correct.

Suggested Build Order

Data model + JPA entities + Oracle schema (DDL/sequences)
Auth (register/login/JWT)
Beneficiary CRUD
Transaction initiate + risk engine (pure logic, unit-testable first)
State machine + scheduled expiry job
WebSocket countdown push + Oracle JET protection-window UI
Maker-checker flow
Preferences + admin policy config
Dispute workflow
Notifications + audit logging + dashboard